

```
In [ ]: import marimo as mo
```

Universidade da Costa Rica | Escola de Engenharia Elétrica

PyX - Série de tutoriais de Python para análise de dados

Py0 - *Introdução ao Python*

Python é uma linguagem de programação de uso geral, atualmente a mais popular para análise de dados. Sua sintaxe foi pensada desde o início para ser mais legível.

Fabián Abarca e Elizabet Medeiros

Instalação

Existem várias formas de execução de Python. Pode ser executado no próprio computador ou em alguma das várias plataformas online que existem, como [Google Colab](#) ou [molab](#). Na máquina local é necessário instalar Python e um gerenciador de pacotes em qualquer plataforma (Linux, Windows, macOS).

- A instalação recomendada de Python e seus pacotes é com [uv](#).
- Também é possível a instalação do [Anaconda](#).

Uma vez instalada alguma dessas ferramentas, é possível "rodar" *scripts* com a extensão `.py` pelo terminal (também chamado de linha de comandos, console ou CLI, de *Command Line Interface*).

- Executando o código-fonte com Python:

```
$ python script.py
```

- Ou usá-lo em modo interativo na CLI:

```
$ python
>>> print('Olá')
Olá
```

ou a partir de qualquer um dos muitos **IDE** (ambientes de desenvolvimento integrado, *Integrated Development Environments*) disponíveis, como:

- [Visual Studio Code](#)

- [Cursor](#)
- [Antigravity](#)
- [Eclipse](#)
- [PyCharm](#)
- [Sublime](#)

Mais informações

As informações mais precisas sobre os aspectos básicos da linguagem Python estão no [manual de referência](#) da *Biblioteca Padrão* do Python. No entanto, é possível encontrar muitas outras boas referências na internet, desde cursos online, páginas de referência (ver final do documento), perguntas em fóruns e até o muito bom [Wikibook de Python](#).

Antes de começar...

Nota 0: Para executar uma célula de código neste *notebook* utilizam-se as teclas `shift` + `enter`, ou "Run" no painel superior quando a célula está selecionada.

Nota 1: A função `print()` mostra o resultado da avaliação do(s) seu(s) argumento(s).

Nota 2: Os comentários no código-fonte de Python são feitos com `#` em uma única linha ou com `''' (comentário) '''` em várias linhas.

Nota 3: Em Python o índice começa em 0.

Nota 4: Como linguagem orientada a objetos, Python utiliza a "notação do ponto": `objeto.atributo` ou `objeto.método()`, que são *variáveis* e *funções* associadas a um objeto.

Nota 5: A forma "pitônica" de programar em Python (*the Pythonic way*) são convenções que tornam o código mais legível e simples.

Nota 6: O *guia de estilo* para a escrita de código em Python é o [PEP 8](#), que estabelece as boas práticas (obrigatórias neste curso) para melhorar a legibilidade e dar consistência ao código.

0.1 - Variáveis

Em Python, a atribuição de um dado a uma variável **não** requer a indicação explícita do *tipo de dado*.

Nota: Isso é conhecido como linguagem "dinamicamente tipada", que verifica o tipo de dado em tempo de execução.

Portanto, basta escrever o seguinte para atribuir números ou caracteres ou qualquer outro objeto de Python a uma variável:

```
In [ ]: number = 15
        string = 'olá!'
        n1, n2, n3 = (1, 2, 3)
        print(number, string, n1 + n2 + n3)
```

0.2 - Valores *booleanos* e comparações

Os dois valores lógicos (*booleanos*) em Python são `True` (ou `1`) e `False` (ou `0`), sobre os quais se aplicam as operações lógicas `or`, `and` e `not`.

```
In [ ]: i = True and (not False)
        j = 0 or 0 or (not 0)
        print(i, j)
```

Comparações entre números

A avaliação das comparações retorna `True` ou `False`.

Operação	Significado
<code><</code>	é estritamente menor que
<code><=</code>	é menor ou igual a
<code>></code>	é estritamente maior que
<code>>=</code>	é maior ou igual a
<code>==</code>	é igual a
<code>!=</code>	não é igual a
<code>is</code>	identidade
<code>is not</code>	identidade negada

Nota 1: `=` é de atribuição, `==` é de comparação.

Nota 2: as operações podem ser encadeadas, por exemplo: `x < y < z`

```
In [ ]: print(12 < 24)
        print(2023 != 2024)
        print(34 >= 21 > 13 > 8)
```

0.3 - Tipos de dados numéricos e seus operadores

Em Python existem três tipos distintos de dados numéricos:

- `int` : número inteiro, positivo ou negativo
- `float` : número de ponto flutuante (decimal) positivo ou negativo
- `complex` : números com uma "parte real" e uma "parte imaginária" de ponto flutuante

Nota 1: As funções respectivas `int()`, `float()` e `complex(a, b)` permitem converter de um tipo para outro.

Nota 2: A função `type()` permite verificar o tipo de dado numérico ou de qualquer dado de uma variável em Python.

```
In [ ]: # Definição
n_int = 2
print(n_int, 'é', type(n_int))
n_float = 3.0
print(n_float, 'é', type(n_float))
n_complex = 5 + 8j
print(n_complex, 'é', type(n_complex))

# Conversão de tipo de dado
new_int = int(n_float)
new_float = float(n_int)
new_complex = complex(new_int, new_float)
print(new_int, new_float, new_complex)
```

Operações básicas sobre dados numéricos

Operação	Resultado
<code>x + y</code>	soma de <code>x</code> e <code>y</code>
<code>x - y</code>	subtração de <code>x</code> e <code>y</code>
<code>x * y</code>	produto de <code>x</code> e <code>y</code>
<code>x / y</code>	quociente de <code>x</code> e <code>y</code>
<code>x // y</code>	piso do quociente de <code>x</code> e <code>y</code>
<code>x % y</code>	resto de <code>x / y</code>
<code>-x</code>	<code>x</code> negativo
<code>+x</code>	<code>x</code> sem mudança de sinal
<code>abs(x)</code>	magnitude de <code>x</code>
<code>int(x)</code>	<code>x</code> convertido para inteiro
<code>float(x)</code>	<code>x</code> convertido para ponto flutuante
<code>complex(re, im)</code>	número complexo

Operação	Resultado
<code>c.conjugate()</code>	conjugado do número complexo <code>c</code>
<code>divmod(x, y)</code>	o par <code>(x // y, x % y)</code>
<code>pow(x, y)</code>	<code>x</code> elevado à potência de <code>y</code>
<code>x ** y</code>	<code>x</code> elevado à potência de <code>y</code>

```
In [ ]: print(10 / 3)
        print(10 // 3)
        print(10 % 3)
        m, n = divmod(10, 3)
        print(m, n, 3 * m + n)
```

0.4 - Números binários, octais, hexadecimais e suas operações

Para declarar um número **binário** inicia-se com `0b`, para declarar um número **octal** inicia-se com `0o`, para declarar um número **hexadecimal** inicia-se com `0x`. Para converter entre um tipo e outro e também `int` utilizam-se as funções `bin()`, `oct()`, `hex()` e `int()`.

```
In [ ]: a = 42
        b = 2020
        c = 25
        print(bin(a))
        print(hex(a))
        print(int(b))
        print(int(c))
```

Operações binárias (bit a bit) sobre números inteiros

Operação	Resultado
<code>x [barra] y ^</code>	OR de <code>x</code> e <code>y</code>
<code>x ^ y</code>	XOR de <code>x</code> e <code>y</code>
<code>x & y</code>	AND de <code>x</code> e <code>y</code>
<code>x << n</code>	Deslocamento à esquerda de <code>x</code> em <code>n</code> bits
<code>x >> n</code>	Deslocamento à direita de <code>x</code> em <code>n</code> bits
<code>~x</code>	NOT (inversão) de <code>x</code>

`^ [barra]` é |

```
In [ ]: a = 21 & 10
        b = ~a
        c = 13 | 21
        d = 74 << 2
        print(a, b, c, d)
```

0.5 - Tipos de sequências de dados e seus operadores

As variáveis do tipo "sequência" permitem armazenar **grupos de dados**. Também são chamadas de "contêineres".

Em Python existem vários tipos:

- `list` : sequência **mutável** (*que pode mudar*) tipicamente utilizada para armazenar elementos *homogêneos* (de um mesmo tipo de dados ou sequências).
- `tuple` : sequência **imutável** (*que não pode mudar*) tipicamente utilizada para armazenar elementos *heterogêneos* (de tipos diferentes).
- `range` : sequência **imutável** de **números** tipicamente utilizada para iterar em um laço.

0.5.1 Como criar sequências

0.5.1.1 Criação de listas

Todas estas opções criam listas:

- `[]` (lista vazia)
- `[a]`, `[a, b, c]`
- `[x for x in iterable]` (*list comprehension*)
- `list()` ou `list(iterable)`

Nota: Assim como em algumas linguagens de computação científica (Matlab, R...), as listas podem ser utilizadas para criar "vetores" e "matrizes".

```
In [ ]: L1 = ['ha']
        L2 = L1 * 3
        L3 = ['alpha', 'beta', 'gamma']
        L4 = [x for x in range(6)]
        L5 = [x for x in range(6) if x % 3 != 0]
        L6 = [[1,2,3],[4,5,6],[7,8,9]]

        print(L1, L2, L3)
        print(L4)
        print(L5)
        print(L6)
```

0.5.1.2 Criação de tuplas

- `()` (tupla vazia)
- `a`, ou `(a,)`
- `a, b, c` ou `(a, b, c)`
- `tuple()` ou `tuple(iterable)`

```
In [ ]: T1 = (1,)
T2 = 1, 2, 3
T3 = ('In', 'a', 'galaxy', 'far', 'far', 'away')
T4 = tuple([x for x in range(3)])

print(type(T1))
print(T2, T3, T4)
```

0.5.1.3 Criação de ranges

- `range(stop)` (começa em 0 e **não** inclui `stop`)
- `range(start, stop[, step])` (como em `start:step:stop`)

```
In [ ]: range(1000)
```

0.5.2 Operações sobre sequências

Operação	Resultado
<code>x in s</code>	<code>True</code> se um elemento de <code>s</code> é igual a <code>x</code> , caso contrário <code>False</code>
<code>x not in s</code>	<code>False</code> se um elemento de <code>s</code> é igual a <code>x</code> , caso contrário <code>True</code>
<code>s + t</code>	a concatenação de <code>s</code> e <code>t</code>
<code>s * n</code> ou <code>n * s</code>	equivalente a adicionar <code>s</code> a si mesmo <code>n</code> vezes
<code>s[i]</code>	<code>i</code> -ésimo elemento de <code>s</code> , com origem em 0
<code>s[i:j]</code>	fatia de <code>s</code> de <code>i</code> a <code>j</code>
<code>s[i:j:k]</code>	fatia de <code>s</code> de <code>i</code> a <code>j</code> com passo <code>k</code>
<code>len(s)</code>	comprimento de <code>s</code>
<code>min(s)</code>	menor elemento de <code>s</code>
<code>max(s)</code>	maior elemento de <code>s</code>
<code>s.index(x[, i[, j]])</code>	índice da primeira ocorrência de <code>x</code> em <code>s</code> (no ou após o índice <code>i</code> e antes do índice <code>j</code>)
<code>s.count(x)</code>	número total de ocorrências de <code>x</code> em <code>s</code>

Nota: A forma "pitônica" de acessar o último elemento da sequência é com o índice `-1`, ou seja, `s[-1]`. Dessa forma não é necessário conhecer *a priori* a quantidade de elementos em `s`.

```
In [ ]: start = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
        continuation = 55, 89, 144, 233
        sequence = start + continuation

        print(sequence)
        print(sequence[4])
        print(sequence[0:-1:2])
        print(sequence.count(1))
```

0.5.3 Operações sobre sequências *mutáveis*

Operação	Resultado
<code>s[i] = x</code>	o elemento <code>i</code> de <code>s</code> é substituído por <code>x</code>
<code>s[i:j] = t</code>	uma porção de <code>s</code> de <code>i</code> a <code>j</code> é substituída pelo conteúdo do iterável <code>t</code>
<code>del s[i:j]</code>	igual a <code>s[i:j] = []</code>
<code>s[i:j:k] = t</code>	os elementos de <code>s[i:j:k]</code> são substituídos pelos de <code>t</code>
<code>del s[i:j:k]</code>	elimina os elementos <code>s[i:j:k]</code> da lista
<code>s.append(x)</code>	adiciona <code>x</code> ao final da sequência (igual a <code>s[len(s):len(s)] = [x]</code>)
<code>s.clear()</code>	elimina todos os elementos de <code>s</code> (igual a <code>del s[:]</code>)
<code>s.copy()</code>	cria uma cópia superficial de <code>s</code> (igual a <code>s[:]</code>)
<code>s.extend(t)</code> ou <code>s += t</code>	estende <code>s</code> com o conteúdo de <code>t</code> (na maior parte igual a <code>s[len(s):len(s)] = t</code>)
<code>s *= n</code>	atualiza <code>s</code> com seu conteúdo repetido <code>n</code> vezes
<code>s.insert(i, x)</code>	insere <code>x</code> em <code>s</code> no índice dado por <code>i</code> (igual a <code>s[i:i] = [x]</code>)
<code>s.pop([i])</code>	recupera o elemento em <code>i</code> e também o remove de <code>s</code>
<code>s.remove(x)</code>	remove o primeiro elemento de <code>s</code> onde <code>s[i]</code> é igual a <code>x</code>
<code>s.reverse()</code>	inverte os elementos de <code>s</code> no lugar

```
In [ ]: phrase = ['all', 'you', 'need', 'is', 'love']
        print(phrase)

        need = 'food'
        phrase[-1] = need
        print(phrase)
```

0.6 - Sequências de texto (*strings*) e seus operadores

As "cadeias" de texto ou *strings* `str` são sequências imutáveis de caracteres alfanuméricos.

0.6.1 Criação de sequências de texto

- `'olá'` (pode conter aspas duplas dentro)
- `"olá"` (pode conter aspas simples dentro)
- `'''olá'''` ou `\"\\\"olá\\\"\\\"` (múltiplas linhas)
- Com a função `str()` a partir de outro tipo de dado

Nota: As sequências de texto permitem todas as operações aplicadas sobre sequências, mas são imutáveis, portanto não admitem as operações sobre sequências mutáveis (como as substituições ou a remoção de elementos).

```
In [ ]: print('Hello "world"' + ", " + '''greetings.''' )
```

0.6.2 Algumas funções ("métodos") sobre cadeias de caracteres

É possível realizar operações específicas de texto sobre um `str`. A lista completa de métodos está disponível [aqui](#).

- `str.capitalize()` : Retorna uma cópia da cadeia com seu primeiro caractere em maiúscula e o restante em minúscula.
- `str.endswith( sufixo [ , início [ , fim ] ] )` : Retorna `True` se a cadeia termina com o sufixo especificado, caso contrário retorna `False`.
- `str.lower()` : Retorna uma cópia da cadeia com todos os caracteres em maiúscula convertidos para minúsculas.

```
In [ ]: text = 'hElLo, gOoD mOrnInG'

print(text)
print(text.capitalize())
print(text.lower().endswith('ing'))
print(text.upper())
```

0.6.3 Formatação de texto ao imprimir

Um [método](#) importante aplicado em cadeias de caracteres permite inserir valores em um texto, com certas especificações.

- `str.format(*args, **kwargs)` : Realiza uma operação de formatação da cadeia. A cadeia para a qual este método é chamado pode conter texto literal ou **campos de substituição** delimitados por chaves `{}` . Cada campo de substituição contém o índice numérico de um argumento posicional ou o nome de um argumento de palavra-chave. `str.format` retorna uma cópia da cadeia onde cada campo de substituição é substituído pelo valor de cadeia do argumento correspondente.

Nota 1: `*args` é um número variável de argumentos que são passados a uma função. Por exemplo, uma função `multiplicar(*args)` que multiplica todos os argumentos passados pode ser `multiplicar(3,3,2,7)` ou `multiplicar(2,3)` . Os `*args` são *posicionais*, ou seja, devem estar na ordem exata em que a função os requer. Por exemplo:

```
c = dividir(a, b)
# c = a/b
```

Nota 2: `**kwargs` é um número variável de argumentos do tipo `chave=argumento` . Os `**kwargs` não são posicionais, podem ser inseridos em qualquer posição.

```
c = dividir(den=b, num=a)
# c = a/b
```

Recomendação: *f-strings*

Uma sintaxe alternativa para incorporar variáveis em um texto é por meio da notação de *f-strings*, que é do tipo:

```
a = 1600
texto = f'Nos anos {a}...
```

```
In [ ]: word = 'world'
x, y = (5, 3)
z = x / y
w = 1600
print('Hello {}'.format(word))
print('{} entre {} é cerca de {:.0.2f}'.format(x, y, z))
print(f'Nos anos {w}...')
```

0.7 - Dicionários

Um dicionário é um tipo especial de sequência mutável que faz um *mapeamento* de valores a uma chave, do tipo `chave: valor` (usualmente em inglês como `key: value`). O nome é uma analogia com os dicionários que têm pares **palavra: definição**.

0.7.1 Criação de dicionários

- Os dicionários podem ser criados colocando uma lista de pares separados por vírgulas entre chaves `{}`.
- Com o construtor `dict()`.

Nota: A ordem dos pares não importa. Se dois ou mais dicionários compartilham exatamente os mesmos pares (mesmo que tenham sido criados em ordem diferente) então são iguais.

```
In [ ]: # Com as chaves
a = {'um': 1, 'dois': 2, 'tres': 3}
b = dict(um=1, dois=2, tres=3)

# Com a função dict()
c = dict(zip(['um', 'dois', 'tres'], [1, 2, 3]))
d = dict([('dois', 2), ('um', 1), ('tres', 3)])

# Com a função zip() que "emparelha" duas listas
e = dict({'tres': 3, 'um': 1, 'dois': 2})
print(a == b == c == d == e)

# Com uma lista de tuplas (ordem diferente)
# Criação redundante, com dict({})
# Verificar igualdade
# Mostrar algum dos dicionários
print(c)
```

0.7.2 Algumas funções úteis em dicionários

- `list(d)` : Retorna uma lista de todas as chaves utilizadas no dicionário `d`.
- `len(d)` : Retorna o número de elementos no dicionário `d`.
- `d[key]` : Retorna o item de `d` com a chave `key`. Gera um `KeyError` se não está no mapa (dicionário).

```
In [ ]: d = {'IE0247': 'Sinais e Sistemas I', 'IE0347': 'Sinais e Sistemas II', 'IE0405': 'Sinais e Sistemas III', 'MA1001': 'Matemática I'}
print(list(d))
print(len(d))
print(d['IE0405'])
print('MA1001' in d)
```

0.8 - Controles de fluxo

Os controles de fluxo determinam as ações do programa diante da avaliação de "declarações" como comparações. A documentação [oficial](#) explica seus detalhes.

0.8.1 Controles de fluxo `if` - `elif` - `else`

Possivelmente o mais importante ou conhecido, `if` avalia uma declaração e prossegue com linhas de execução distintas conforme o resultado. Em Python sua sintaxe é:

```
if <declaracao>:  
    <acao quando eh verdadeiro>
```

Quando o resultado é `False` pode haver outra ação possível, e a sintaxe é:

```
if <declaracao>:  
    <acao quando eh verdadeiro>  
else:  
    <acao quando eh falso>
```

Se ao avaliar a primeira declaração há outras possibilidades então devem ser avaliadas outras declarações com a sintaxe:

```
if <declaracao_1>:  
    <acao quando 1 eh verdadeiro>  
elif <declaracao_2>:  
    <acao quando 1 eh falso e 2 eh verdadeiro>  
else:  
    <acao quando 1 e 2 sao falsos>
```

Nota: a *indentação* é **obrigatória** em Python, e é também suficiente para delimitar o que está contido dentro de um curso de ação. Por exemplo, no seguinte fragmento, `print(a)` está dentro do `if` e `print(b)` não.

```
if a < b:  
    print(a)
```

```
print(b)
```

Nota: este exemplo inclui uma **biblioteca** de Python (`datetime`) para obter a hora atual. O uso e importação de bibliotecas é apresentado em `Py1` .

```
In [ ]: import datetime  
  
# Hora atual  
H = datetime.datetime.now().hour  
  
# Saudação conforme a hora do dia  
if H < 12:  
    print('Bom dia')  
elif H < 18:  
    print('Boa tarde')  
else:  
    print('Boa noite')
```

0.8.2 Controles de fluxo `try` - `except` para tratamento de erros

Python encerra a execução de um programa quando encontra um erro. Isso diferente de uma linguagem compilada (como C e C++) que não executa o programa até descartar

erros em uma compilação prévia.

Frequentemente é necessário evitar que o programa **não** pare quando há um erro.

Quando uma instrução pode conter um erro com certa probabilidade, é possível fazer um teste com a instrução `try`.

O comando `except` permite agir em caso de erro.

Outras opções com `else` e `finally` podem ser encontradas [aqui](#).

```
In [ ]: # x = 3

# Se a definição de x está "comentada"
try:
    print(f'A variável existe e é {x}.')
except:
    print('Opa! A variável não existe')

# Agora pode-se testar "descomentando" x
```

0.8.3 Controle de fluxo `match – case`

Uma alternativa para usar uma lógica `if` no caso em que há várias opções possíveis, é a comparação de uma variável de interesse com seus valores possíveis e executar ações para cada caso, como:

```
match variable:
    case "a":
        print("It's a!")
    case "b":
        print("It's b!")
    case "c":
        print("It's c!")
    case _:
        print("It was something else")
```

```
In [ ]: variable = "a"

match variable:
    case "a":
        print("It's 'a'!")
    case "b":
        print("It's 'b'!")
    case "c":
        print("It's 'c'!")
    case _:
        print("It was something else")
```

0.9 - Laços

Um tipo especial de controles de fluxo são os que **repetem** uma ação em "laço" ou "loop" (*loop*).

0.9.1 Laço `while`

Repete uma operação **indefinidamente** enquanto uma condição se cumpra.

```
while <declaracao>:  
    <acao enquanto for verdadeiro>
```

0.9.2 Laço `for`

Repete uma operação por **um número determinado de vezes**, ao iterar *sobre* uma sequência de dados.

```
for i in (a, b, c, d, e, f):  
    <acao para i = a, depois i = b, depois i = c...>
```

Nota 1: a sequência `range()` é útil quando se quer iterar sobre uma longa sequência de números (por exemplo, de 10 até 100).

Nota 2: O operador `a += n` é equivalente a `a = a + n`. Também existem `-=` e `*=`.

```
In [ ]: print('Primeiro:')  
        i = 1  
        while i % 5 != 0:  
            print(i)  
            i += 1  
  
        print('Segundo:')  
        for j in (8, 13, 21, 34):  
            print(j)  
  
        print('Terceiro:')  
        for k in range(5):  
            print(k)
```

Outros comandos especiais

- `break` : Sai do ciclo atual.
- `yield` : Continua com a próxima iteração do laço.

0.10 - Algumas funções nativas

Já são conhecidas algumas funções da [Biblioteca Padrão do Python](#) como `print()`, `type()`, `int()`, `len()`, `list()`. Algumas outras funções úteis de interesse são:

- `abs(a)` : Valor absoluto de um número.
- `all()` : `True` se todos os elementos de uma sequência são `True`.
- `enumerate(iterable[, start=n])` : "adere" numeração a uma sequência iterável, começando em `start=n` (por padrão `start=0`).
- `help()` : Invoca a ajuda de um objeto (função ou comando ou variável).
- `input([prompt])` : Habilita a entrada de dados com um texto `[prompt]` quando o programa é executado em um terminal de comandos ou aqui no Jupyter também.
- `map(função, iterable)` : Aplica a `função` sobre cada elemento de `iterable`.
- `round(n[, dígitos])` : Arredonda `n` com uma precisão de `dígitos`.
- `zip(*iterables)` : Cria um novo iterador com a união de tantos `*iteradores` quanto forem adicionados. O `i`-ésimo elemento de `zip(*iterables)` é uma tupla com os `i`-ésimos elementos de cada argumento iterável.

Nota: O Jupyter às vezes não exibe a entrada de dados. Para isso pode-se tentar no menu Kernel > Restart (as variáveis armazenadas serão apagadas).

```
In [ ]: name = input('Digite seu nome: ')

print('Hi {}'.format(name))
print('This is the help for function "abs":\n')
help(abs)
```

Isso é tudo por agora!

Os conceitos vistos até agora em Py0 são poderosos e permitem a programação de soluções para uma grande quantidade de problemas computacionais. Mesmo em tarefas muito mais complexas no futuro, essas bases continuarão sendo essenciais.

No entanto, o verdadeiro segredo do Python são os muitos pacotes que lhe dão "poderes especiais" para quase todo tipo de tarefas, desenvolvidos por uma comunidade ativa ao redor do mundo.

O restante dos PyX será um guia introdutório para algumas dessas ferramentas em [computação científica](#).

Filosofia do Python

Este "zen" é uma coleção de princípios de programação (ou de vida?) que guiam as boas práticas em Python.

- Bonito é melhor que feio.
- Explícito é melhor que implícito.
- Simples é melhor que complexo.
- Complexo é melhor que complicado.
- Plano é melhor que aninhado.
- Esparsos são melhores que densos.
- A legibilidade conta.
- Os casos especiais não são especiais o bastante para quebrar as regras.
- O prático vence o puro.
- Os erros nunca devem passar silenciosamente.
- A menos que sejam explicitamente silenciados.
- Diante da ambiguidade, recuse a tentação de adivinhar.
- Deveria haver uma — e preferencialmente apenas uma — maneira óbvia de fazer.
- Embora essa maneira possa não ser óbvia à primeira vista, a menos que você seja holandês.
- Agora é melhor que nunca.
- Embora nunca frequentemente seja melhor que *agora mesmo*.
- Se a implementação é difícil de explicar, é uma má ideia.
- Se a implementação é fácil de explicar, pode ser uma boa ideia.
- Os espaços de nomes (*namespaces*) são uma grande ideia, vamos fazer mais!

Nota: Um *namespace* é uma forma de garantir que não há nomes de variáveis ou funções ou métodos repetidos entre o *script* e as bibliotecas e módulos utilizados.

```
In [ ]: # Estes princípios estão escondidos aqui:
import this
```

Mais informações

- [Documentação oficial do Python](#)
- [RealPython](#)
- [w3schools](#)
- [Wikibook de Python](#)
- (...muitas outras referências na web e livros...)

